8086 Assembly Branch Instructions Practice Problems

Branching and conditional logic using DOS interrupts (INT 21H)

Assembly Branch Instructions Practice Problems

Problem 1: Simple Character Comparison

Question: Write an assembly program that prompts the user to enter a character. If the character is 'A', display "CORRECT!". Otherwise, display "WRONG!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A CHARACTER: $'
CORRECT_MSG DB 'CORRECT!$'
WRONG_MSG DB 'WRONG!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'A'
    JE PRINT_CORRECT

    LEA DX,WRONG_MSG
    JMP PRINT_MSG

    PRINT_CORRECT:
    LEA DX,CORRECT_MSG

    PRINT_MSG:
```

```
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 2: Number Range Check

Question: Write a program that inputs a digit (0-9) and displays "SMALL" if it's 0-4, "LARGE" if it's 5-9, or "INVALID" if it's not a digit.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
SMALL_MSG DB 'SMALL$'
LARGE_MSG DB 'LARGE$'
INVALID_MSG DB 'INVALID$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'0'
    JL INVALID
    CMP AL,'9'
    JG INVALID
    CMP AL,'5'
    JL SMALL

    LEA DX,LARGE_MSG
    JMP DISPLAY

    SMALL:
    LEA DX,SMALL_MSG
    JMP DISPLAY
```

```
    INVALID:
    LEA DX,INVALID_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 3: Vowel or Consonant

Question: Write a program that checks if an entered character is a vowel (A,E,I,O,U) or consonant. Display appropriate message.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A CHARACTER: $'
VOWEL_MSG DB 'VOWEL$'
CONSONANT_MSG DB 'CONSONANT$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'A'
    JE VOWEL
    CMP AL,'E'
    JE VOWEL
    CMP AL,'I'
    JE VOWEL
    CMP AL,'O'
    JE VOWEL
    CMP AL,'U'
    JE VOWEL
```

```
    LEA DX,CONSONANT_MSG
    JMP DISPLAY

    VOWEL:
    LEA DX,VOWEL_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 4: Even or Odd Number

Question: Write a program that inputs a single digit and determines if it's even or odd using branch instructions.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
EVEN_MSG DB 'EVEN$'
ODD_MSG DB 'ODD$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    AND AL,1          ; Check least significant bit
    JZ EVEN           ; If zero, number is even

    LEA DX,ODD_MSG
    JMP DISPLAY

    EVEN:
```

```
    LEA DX,EVEN_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 5: Password Check

Question: Write a program that prompts for a password. If the user enters 'X', display "ACCESS GRANTED". Otherwise, display "ACCESS DENIED".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER PASSWORD: $'
GRANTED_MSG DB 'ACCESS GRANTED$'
DENIED_MSG DB 'ACCESS DENIED$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'X'
    JNE ACCESS_DENIED

    LEA DX,GRANTED_MSG
    JMP DISPLAY

    ACCESS_DENIED:
    LEA DX,DENIED_MSG

    DISPLAY:
    MOV AH,9
    INT 21H
```

```
    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 6: Grade Classification

Question: Write a program that inputs a character grade (A,B,C,D,F) and displays the corresponding message: "EXCELLENT" for A, "GOOD" for B, "AVERAGE" for C, "POOR" for D, and "FAIL" for F.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER GRADE: $'
EXCELLENT_MSG DB 'EXCELLENT$'
GOOD_MSG DB 'GOOD$'
AVERAGE_MSG DB 'AVERAGE$'
POOR_MSG DB 'POOR$'
FAIL_MSG DB 'FAIL$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'A'
    JE EXCELLENT
    CMP AL,'B'
    JE GOOD
    CMP AL,'C'
    JE AVERAGE
    CMP AL,'D'
    JE POOR
    CMP AL,'F'
    JE FAIL
    JMP EXIT

    EXCELLENT:
    LEA DX,EXCELLENT_MSG
```

```
    JMP DISPLAY

    GOOD:
    LEA DX,GOOD_MSG
    JMP DISPLAY

    AVERAGE:
    LEA DX,AVERAGE_MSG
    JMP DISPLAY

    POOR:
    LEA DX,POOR_MSG
    JMP DISPLAY

    FAIL:
    LEA DX,FAIL_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    EXIT:
    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 7: Upper/Lower Case Check

Question: Write a program that determines if an entered character is uppercase, lowercase, or neither.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A CHARACTER: $'
UPPER_MSG DB 'UPPERCASE$'
LOWER_MSG DB 'LOWERCASE$'
NEITHER_MSG DB 'NEITHER$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H
```

```
    MOV AH,1
    INT 21H

    CMP AL,'A'
    JL CHECK_LOWER
    CMP AL,'Z'
    JG CHECK_LOWER

    LEA DX,UPPER_MSG
    JMP DISPLAY

    CHECK_LOWER:
    CMP AL,'a'
    JL NEITHER
    CMP AL,'z'
    JG NEITHER

    LEA DX,LOWER_MSG
    JMP DISPLAY

    NEITHER:
    LEA DX,NEITHER_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 8: Simple Calculator Choice

Question: Write a program that displays a menu (1-Add, 2-Subtract, 3-Exit) and branches to appropriate sections based on user choice.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MENU DB '1-ADD, 2-SUBTRACT, 3-EXIT: $'
ADD_MSG DB 'ADDITION SELECTED$'
SUB_MSG DB 'SUBTRACTION SELECTED$'
EXIT_MSG DB 'EXITING...$'

.CODE
```

```
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MENU
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'1'
    JE ADD_OPTION
    CMP AL,'2'
    JE SUB_OPTION
    CMP AL,'3'
    JE EXIT_OPTION
    JMP END_PROGRAM

    ADD_OPTION:
    LEA DX,ADD_MSG
    JMP DISPLAY

    SUB_OPTION:
    LEA DX,SUB_MSG
    JMP DISPLAY

    EXIT_OPTION:
    LEA DX,EXIT_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    END_PROGRAM:
    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 9: Number Comparison

Question: Write a program that inputs two single digits and displays which one is larger, or if they are equal.

Answer:

```
.MODEL SMALL
.STACK 100H
```

```
.DATA
MSG1 DB 'ENTER FIRST DIGIT: $'
MSG2 DB 'ENTER SECOND DIGIT: $'
FIRST_MSG DB 'FIRST IS LARGER$'
SECOND_MSG DB 'SECOND IS LARGER$'
EQUAL_MSG DB 'BOTH ARE EQUAL$'
NUM1 DB ?
NUM2 DB ?

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    MOV NUM1,AL

    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    MOV NUM2,AL

    MOV AL,NUM1
    CMP AL,NUM2
    JG FIRST_LARGER
    JL SECOND_LARGER

    LEA DX,EQUAL_MSG
    JMP DISPLAY

    FIRST_LARGER:
    LEA DX,FIRST_MSG
    JMP DISPLAY

    SECOND_LARGER:
    LEA DX,SECOND_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 10: Loop with Condition

Question: Write a program that repeatedly asks for a character until the user enters 'Q' to quit. Display "CONTINUE" for any other character.

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER CHARACTER (Q to quit): $'
CONTINUE_MSG DB 'CONTINUE', 0DH, 0AH, '$'
QUIT_MSG DB 'GOODBYE!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    INPUT_LOOP:
    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H

    CMP AL,'Q'
    JE QUIT

    LEA DX,CONTINUE_MSG
    MOV AH,9
    INT 21H

    JMP INPUT_LOOP

    QUIT:
    LEA DX,QUIT_MSG
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Simple Assembly Branch Instructions Practice Problems

Problem 1: Basic Addition Check

Question: Write a program that adds two single digits and checks if the result is greater than 10. Display "BIG SUM!" if result > 10, otherwise display "SMALL SUM!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER FIRST DIGIT: $'
MSG2 DB 'ENTER SECOND DIGIT: $'
BIG_MSG DB 'BIG SUM!$'
SMALL_MSG DB 'SMALL SUM!$'
NUM1 DB ?
NUM2 DB ?

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number
    MOV NUM1,AL

    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number
    MOV NUM2,AL

    MOV AL,NUM1
    ADD AL,NUM2       ; Add the numbers
    CMP AL,10         ; Compare with 10
    JG BIG_SUM        ; Jump if greater than 10

    LEA DX,SMALL_MSG
    JMP DISPLAY

    BIG_SUM:
```

```
    LEA DX,BIG_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 2: Even or Odd using AND Logic

Question: Input a digit and use AND operation to check if it's even or odd. Display "EVEN NUMBER!" or "ODD NUMBER!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
EVEN_MSG DB 'EVEN NUMBER!$'
ODD_MSG DB 'ODD NUMBER!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    AND AL,1          ; Check least significant bit
    JZ EVEN           ; If zero, number is even

    LEA DX,ODD_MSG
    JMP DISPLAY

    EVEN:
    LEA DX,EVEN_MSG

    DISPLAY:
    MOV AH,9
```

```
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 3: Simple Subtraction Check

Question: Input two digits. Subtract second from first. If result is positive, display "POSITIVE!". If zero, display "EQUAL!". If negative, display "NEGATIVE!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER FIRST DIGIT: $'
MSG2 DB 'ENTER SECOND DIGIT: $'
POSITIVE_MSG DB 'POSITIVE!$'
EQUAL_MSG DB 'EQUAL!$'
NEGATIVE_MSG DB 'NEGATIVE!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H
    MOV BL,AL         ; Store first number

    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H

    SUB BL,AL         ; BL = first - second
    JZ EQUAL          ; Jump if zero
    JS NEGATIVE       ; Jump if negative (sign flag set)
```

```
    LEA DX,POSITIVE_MSG
    JMP DISPLAY

    EQUAL:
    LEA DX,EQUAL_MSG
    JMP DISPLAY

    NEGATIVE:
    LEA DX,NEGATIVE_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 4: Division by 2 Check

Question: Input a digit. Divide it by 2 using shift operation. If result is greater than 3, display "BIG HALF!", otherwise "SMALL HALF!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
BIG_MSG DB 'BIG HALF!$'
SMALL_MSG DB 'SMALL HALF!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    SHR AL,1          ; Divide by 2 (shift right)
    CMP AL,3          ; Compare with 3
    JG BIG_HALF       ; Jump if greater than 3
```

```
    LEA DX,SMALL_MSG
    JMP DISPLAY

    BIG_HALF:
    LEA DX,BIG_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 5: Bit Pattern Check using OR

Question: Input a digit. OR it with 5. If result is greater than 7, display "HIGH BITS!", otherwise "LOW BITS!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
HIGH_MSG DB 'HIGH BITS!$'
LOW_MSG DB 'LOW BITS!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    OR AL,5           ; OR with 5
    CMP AL,7          ; Compare with 7
    JG HIGH_BITS      ; Jump if greater than 7

    LEA DX,LOW_MSG
    JMP DISPLAY
```

```
    HIGH_BITS:
    LEA DX,HIGH_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 6: Double and Compare

Question: Input a digit. Multiply it by 2 (using ADD or SHL). If result is less than 10, display "FITS!", otherwise "TOO BIG!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
FITS_MSG DB 'FITS!$'
BIG_MSG DB 'TOO BIG!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    SHL AL,1          ; Multiply by 2 (shift left)
    CMP AL,10         ; Compare with 10
    JL FITS           ; Jump if less than 10

    LEA DX,BIG_MSG
    JMP DISPLAY

    FITS:
    LEA DX,FITS_MSG
```

```
    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 7: XOR Pattern Check

Question: Input a digit. XOR it with 3. If result is 0, display "MAGIC NUMBER!", otherwise display "NORMAL NUMBER!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
MAGIC_MSG DB 'MAGIC NUMBER!$'
NORMAL_MSG DB 'NORMAL NUMBER!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    XOR AL,3          ; XOR with 3
    JZ MAGIC          ; Jump if zero

    LEA DX,NORMAL_MSG
    JMP DISPLAY

    MAGIC:
    LEA DX,MAGIC_MSG

    DISPLAY:
    MOV AH,9
    INT 21H
```

```
    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 8: Simple Counter Loop

Question: Start with number 5. Keep subtracting 1 and display count until it reaches 0. When 0, display "COUNTDOWN FINISHED!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
COUNT_MSG DB 'COUNT: $'
FINISH_MSG DB 'COUNTDOWN FINISHED!$'
NEWLINE DB 0DH,0AH,'$'
COUNTER DB 5

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    COUNT_LOOP:
    CMP COUNTER,0     ; Check if counter is 0
    JE FINISHED       ; Jump if zero

    LEA DX,COUNT_MSG
    MOV AH,9
    INT 21H

    MOV DL,COUNTER
    ADD DL,30H        ; Convert to ASCII
    MOV AH,2
    INT 21H

    LEA DX,NEWLINE
    MOV AH,9
    INT 21H

    DEC COUNTER       ; Subtract 1
    JMP COUNT_LOOP    ; Repeat loop

    FINISHED:
    LEA DX,FINISH_MSG
    MOV AH,9
```

```
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

Problem 9: NOT Operation Check

Question: Input a digit. Apply NOT operation and check if result is greater than 240. Display "BIG NOT!" or "SMALL NOT!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER A DIGIT: $'
BIG_MSG DB 'BIG NOT!$'
SMALL_MSG DB 'SMALL NOT!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H        ; Convert to number

    NOT AL            ; Apply NOT operation
    CMP AL,240        ; Compare with 240
    JG BIG_NOT        ; Jump if greater

    LEA DX,SMALL_MSG
    JMP DISPLAY

    BIG_NOT:
    LEA DX,BIG_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
```

```
    INT 21H
MAIN ENDP
END MAIN
```

Problem 10: Simple Average Check

Question: Input two digits. Add them and divide by 2 (using SHR). If average is 4 or more, display "GOOD AVERAGE!", otherwise "LOW AVERAGE!".

Answer:

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'ENTER FIRST DIGIT: $'
MSG2 DB 'ENTER SECOND DIGIT: $'
GOOD_MSG DB 'GOOD AVERAGE!$'
LOW_MSG DB 'LOW AVERAGE!$'

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H
    MOV BL,AL         ; Store first number

    LEA DX,MSG2
    MOV AH,9
    INT 21H

    MOV AH,1
    INT 21H
    SUB AL,30H

    ADD AL,BL         ; Add both numbers
    SHR AL,1          ; Divide by 2 (average)
    CMP AL,4          ; Compare with 4
    JGE GOOD_AVG      ; Jump if greater or equal

    LEA DX,LOW_MSG
    JMP DISPLAY
```

```
    GOOD_AVG:
    LEA DX,GOOD_MSG

    DISPLAY:
    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```